

ФЕДЕРАЛЬНОЕ ГОСУДАРСТВЕННОЕ АВТОНОМНОЕ ОБРАЗОВАТЕЛЬНОЕ УЧРЕЖДЕНИЕ
ВЫСШЕГО ОБРАЗОВАНИЯ

«Национальный исследовательский университет ИТМО»

Факультет программной инженерии и компьютерной техники

«Алгоритмы и структуры данных»

отчет по блоку задач №1 (Яндекс.Контест)

Выполнил:

Студент группы Р3210

Давидюк Антон

Задача №1 «А. Агроном-любитель»

Разбор:

end_streak перебирает все цветки, это правый конец серии подходящих грядок, start_streak это левый конец серии подходящих грядок. Алгоритм начинает работу с первого цветка, при добавлении нового цветка проверяется условие: если текущий цветок и два предыдущих одинаковы, то в серии 3 одинаковых цветка подряд. В этом случае левый конец start_streak сдвигается на end_streak-1. Так исключается первый цветок из тройки, но остается два одинаковых. На каждом шаге вычисляется длина текущего участка, если она больше прошлой максимальной серии грядок max_streak_len, обновляются значения максимальной длины и границы максимальной серии (max_streak_left, max_streak_right). Проходя по всем цветкам один раз, мы убеждаемся в отсутствии трех одинаковых цветов в серии и записываем максимум.

Алгоритмическая сложность:

1. По времени: Алгоритм выполняет один проход по массиву, внутри цикла сравнение и присваивание происходит за константное время. Количество операций n равно количеству цветов n , это дает линейную временную сложность $O(n)$.
2. По памяти: Вектор gryadka используется для хранения n входных значений и использует переменные для логики (start_streak, max_streak_len, end_streak и др) Объем используемой памяти зависит от размера входных данных т.к мы используем вектор. Сложность по памяти $O(n)$.

Код:

```
#include <algorithm>
#include <iostream>
#include <vector>

int main() {
    int n;
    int start_streak = 0, max_streak_len = 0;
    int max_streak_left = 1, max_streak_right = 1;
    std::cin >> n;

    std::vector<int> gryadka(n);
    for (int i = 0; i < n; ++i) {
        std::cin >> gryadka[i];
    }

    for (int end_streak = 0; end_streak < n; ++end_streak) {
        if (end_streak >= 2 and gryadka[end_streak] == gryadka[end_streak - 1] and
            gryadka[end_streak] == gryadka[end_streak - 2]) {

            start_streak = std::max(start_streak, end_streak - 1);
        }

        int current_len = end_streak - start_streak + 1;

        if (current_len > max_streak_len) {
            max_streak_len = current_len;
            max_streak_left = start_streak + 1;
            max_streak_right = end_streak + 1;
        }
    }

    std::cout << max_streak_left << " " << max_streak_right << std::endl;
    return 0;
}
```

Задача №2 «В. Зоопарк Глеба»

Не смог решить :(, код не прошел все проверки.

Задача №3 «С. Конфигурационный файл»

Разбор:

При открытии блока { размер истории hist сохраняется в стек blocks. При обработке left=right алгоритм сначала сохраняет значение left из mem в hist, после mem получает новое значение. При закрытии блока } из blocks берется контрольная точка и восстанавливаются значения переменных, удаляются записи из hist пока не придем к сохраненному размеру, старые значения возвращаются в mem. После выхода из блока переменные возвращаются к прежним значениям. Перед выводом происходит проверка $\langle \text{variable1} \rangle = \langle \text{variable2} \rangle$.

Алгоритмическая сложность:

1. По времени: Программа обрабатывает введенные данные один раз в цикле while. Операции с unordered_map поиск и вставка выполняются в среднем за $O(1)$. Операции со стеками hist и blocks добавление и удаление элементов тоже выполняются за $O(1)$. Каждая запись в историю добавляется один раз и удаляется максимум один раз, когда блок закрывается. Временная сложность равна количеству входных данных $O(n)$.
2. По памяти: структуры данных: unordered_map mem, вектор hist ,стек blocks. В худшем случае размер hist и blocks может быть пропорционален кол-ву операций. Размер mem ограничен кол-вом уникальных имен переменных. Эти величины не больше введенных данных n. Сложность по памяти $O(n)$.

Код:

```
#include <iostream>
#include <string>
#include <unordered_map>
#include <vector>

using namespace std;

struct Entry {
    string name;
    long long val;
};

int main() {
    unordered_map<string, long long> mem;
    vector<Entry> hist;
    vector<size_t> blocks;
    string s;

    while (cin >> s) {
        if (s == "{") {
            blocks.push_back(hist.size());
        }
        else if (s == "}") {
```

```

size_t limit = blocks.back();
blocks.pop_back();
while (hist.size() > limit) {
    Entry e = hist.back();
    hist.pop_back();
    mem[e.name] = e.val;
}
}
else {
    size_t eq = s.find('=');
    string left = s.substr(0, eq);
    string right = s.substr(eq + 1);

    long long new_val;
    bool is_var = false;

    if (right[0] >= 'a' && right[0] <= 'z') {
        new_val = mem[right];
        is_var = true;
    }
    else {
        new_val = stoll(right);
    }

    if (is_var) {
        cout << new_val << "\n";
    }

    hist.push_back({ left, mem[left] });
    mem[left] = new_val;
}
}
return 0;
}

```

Задача №4 «D. Профессор Хаос»

Разбор:

История сохраняется в вектор `hist` и записывается день новой записи в `seen`. Сначала популяция равна `a`, т.е. день 0. На каждом шаге вычисляется новое значение по формуле: $cur = \min(cur * b - c, d)$. Если $cur * b < c$, популяция вымирает. Длина цикла это разница между текущим днём и днём первого появления значения. Оставшееся количество дней `k-day` сокращается по модулю длины цикла, это определяют значение на `k`й день. Если не нашли цикл за `k` дней, выводится последнее значение. Состояние популяции ограничено $[0, d]$, так мы либо найдем цикл либо найдем решение за конечное кол-во шагов.

Алгоритмическая сложность:

1. По времени: В худшем случае алгоритм выполняет `k` итераций, но фактически количество упирается в состояния популяции, которое не больше `d + 1` при обнаружении цикла вычисления завершаются. Все операции внутри цикла выполняются за $O(1)$. Временная сложность составляет $O(\min(k, d))$.
2. По памяти: В программе есть два вектора `seen` размером `d + 1` и `hist` размер которого не более `d+1` до обнаружения цикла. Сложность по памяти $O(d)$

Код:

```
#include <iostream>
#include <vector>

using namespace std;

int main() {

    long long a, b, c, d, k;
    cin >> a >> b >> c >> d >> k;

    vector<int> seen(d + 1, -1);
    vector<long long> hist;
    hist.push_back(a);
    seen[a] = 0;

    long long cur = a;
    for (long long day = 1; day <= k; ++day) {
        long long grown = cur * b;

        if (grown < c) {
            cout << 0 << "\n";
            return 0;
        }

        long long after = grown - c;
        cur = (after > d) ? d : after;

        if (seen[cur] != -1) {
            int first = seen[cur];
            long long len = day - first;
            long long rem = k - day;
            long long idx = first + (rem % len);
            cout << hist[idx] << "\n";
            return 0;
        }
        seen[cur] = day;
        hist.push_back(cur);
    }

    cout << cur << endl;
    return 0;
}
```